

1) Break down each opcode's purpose

1. OP_DUP

- Action: duplicates the top element of the stack.
- Typical use: duplicate the public key so one copy can be hashed and compared while the other remains for signature verification.

2. OP_HASH160

- Action: pop the top stack element, compute **RIPEND160(SHA256(x))** (Bitcoin's HASH160), and push that 20-byte digest.
- Typical use: compare a public key's HASH160 to the address's embedded hash.

3. OP_EQUALVERIFY

- Action: pop top two elements, check they are equal; if not, script fails immediately. If equal, continues but pops the compared values (so they are removed).
- Typical use: verify that the HASH160(pubkey) equals the expected value encoded in the scriptPubKey (i.e. address check).

4. OP_CHECKSIG

- Action: pop a signature and a public key, verify the signature is valid for the transaction and public key; pushes true (1) if signature ok, false (0) otherwise.
- Typical use: ensures the spender controls the private key corresponding to the public key.

2)Diagram showing data flow (ASCII)

[scriptSig pushes]	[scriptPubKey]
<signature>	OP_DUP
<pubkey>	OP_HASH160
	<pubKeyHash_expected>

OP_EQUALVERIFY OP_CHECKSIG

Execution (simplified):

Initial stack (top -> bottom):

top -> [<signature>, <pubkey>]

OP_DUP:

-> duplicate pubkey

stack -> [<signature>, <pubkey>, <pubkey>]

OP_HASH160:

-> hash top pubkey

stack -> [<signature>, <pubkey>, hash160(<pubkey>)]

(push expected hash)

stack -> [<signature>, <pubkey>, hash160(<pubkey>), <pubKeyHash_expected>]

OP_EQUALVERIFY:

-> compare hash160(<pubkey>) == <pubKeyHash_expected>

-> if equal pop both and continue; else FAIL

stack -> [<signature>, <pubkey>]

OP_CHECKSIG:

-> verify signature with pubkey against tx

-> pushes true if valid -> Script success

3) What happens if signature verification fails?

OP_CHECKSIG pushes a false (0) on the stack.

Because OP_CHECKSIG is the terminal predicate here, a false result means the script finishes with a false/empty truthy result → the spending transaction is invalid and the node will reject it.

No funds are transferred; the network will not accept the spending transaction into the mempool or blocks.

Concrete consequences:

The spending transaction will be rejected by your node (RPC sendrawtransaction will fail or it will be dropped from mempool).

If the transaction were somehow included in a mined block (it won't be validated by full nodes), chain would consider it invalid — miners and nodes enforce script rules so it will not propagate/confirm.

4)Security benefits of hash verification (OP_HASH160 + OP_EQUALVERIFY)

OP_HASH160 + OP_EQUALVERIFY ensure two-layer protection before signature verification:

1. Smaller on-chain footprint for addresses

- The script stores a 20-byte hash (HASH160(pubkey)) instead of the full public key (33 or 65 bytes). This saves space in UTXO set and common scriptPubKeys.

2. Key-revealing only when spent

- The actual pubkey is revealed only when the UTXO is spent (scriptSig pushes pubkey). Until spending time, only the 20-byte hash is on chain — reduces attacker information.

3. Prevents trivial script substitution

- Even if someone tries to substitute a different pubkey, **OP_EQUALVERIFY** blocks that. The spender must provide a pubkey that hashes to the expected value.

4. Mitigates preimage / quantum discussions

- With hash-based address, the attacker must find a pubkey whose hash equals the stored hash — because HASH160 is collision-resistant, that's infeasible. (Hash-based hiding is a classical defense; note: if the public key is revealed after spending, post-quantum concerns still exist — but still better than exposing pubkey forever.)